

AI for Medicine – Chest-Xray Classification

Prof. Stefano Diciotti – University of Bologna

Academic Year: 2025-2026

Student Name: Reza Farjadnia

Degree Program: Electronics for intelligent systems, big-data and internet of things

Submission Date: 23/06/2026

1. Project Title

Chest X-Ray Classification Using Deep Learning for COVID-19 and Pulmonary Disease Detection

2. Problem Statement

Describe the medical or biological problem addressed by this project. Why is it relevant?

What is the scientific or clinical motivation?

Diseases such as COVID-19, viral pneumonia, and lung opacity can be detected using chest X-ray images. However, examining these images requires time and medical expertise. The motivation of this project is to explore whether **deep learning models can automatically classify chest X-rays** into different disease categories. Such systems could help support radiologists and improve the speed of diagnosis.

3. Objective of the Study

State the specific objective. What are you trying to predict, classify, segment, or analyze?

The main objective of this project is to develop and evaluate a deep learning model capable of automatically classifying chest X-ray images into four disease categories. The study compares a custom Convolutional Neural Network (CNN) with a transfer learning approach based on MobileNet and evaluates their performance using cross-validation and independent test data.

4. Dataset Description

Name and source of the dataset (e.g., MedMNIST, PhysioNet, Kaggle).

- *Number of samples*
- *Type (image, signals, tabular data) and description of data*
- *Available labels (diagnosis, classes, target variables)*
- *Issues such as class imbalance or noise*

The dataset used in this project is the **COVID-19 Radiography Database** from Kaggle. It consists of **21,111 chest X-ray images** and corresponding metadata files containing image paths and class labels. The dataset is an **image dataset**, where the target variable is the disease category associated with each X-ray image. A challenge of this dataset is **class imbalance**, as some classes contain significantly more samples than others.

5. Data Preprocessing

Detail how the data was processed before training the model:

- *Data cleaning, normalization, dimensionality reduction, etc*

Several preprocessing steps were applied before model training. Meta Files Duplicate images were identified and removed to prevent data leakage. All images were resized to 224×224 pixels and converted to three-channel RGB format to match the MobileNet input requirements. Pixel values were normalized to the range [0,1]. Label encoding was performed to convert class names into numerical labels suitable for model training.

6. Avoiding Data Leakage

Explain how the pipeline avoids data leakage. For example:

- *Proper separation of training and test data*
- *Feature selection and normalization are performed within the training folds*
- *No peeking at the test set during model design or tuning*

To ensure a reliable evaluation, duplicate images were removed before splitting the dataset. Stratified train-test splitting was performed to maintain class balance. Furthermore, model selection was carried out using stratified 5-fold cross-validation exclusively on the training data. The independent test set was used only for the final evaluation after model selection was completed.

7. Machine Learning Pipeline

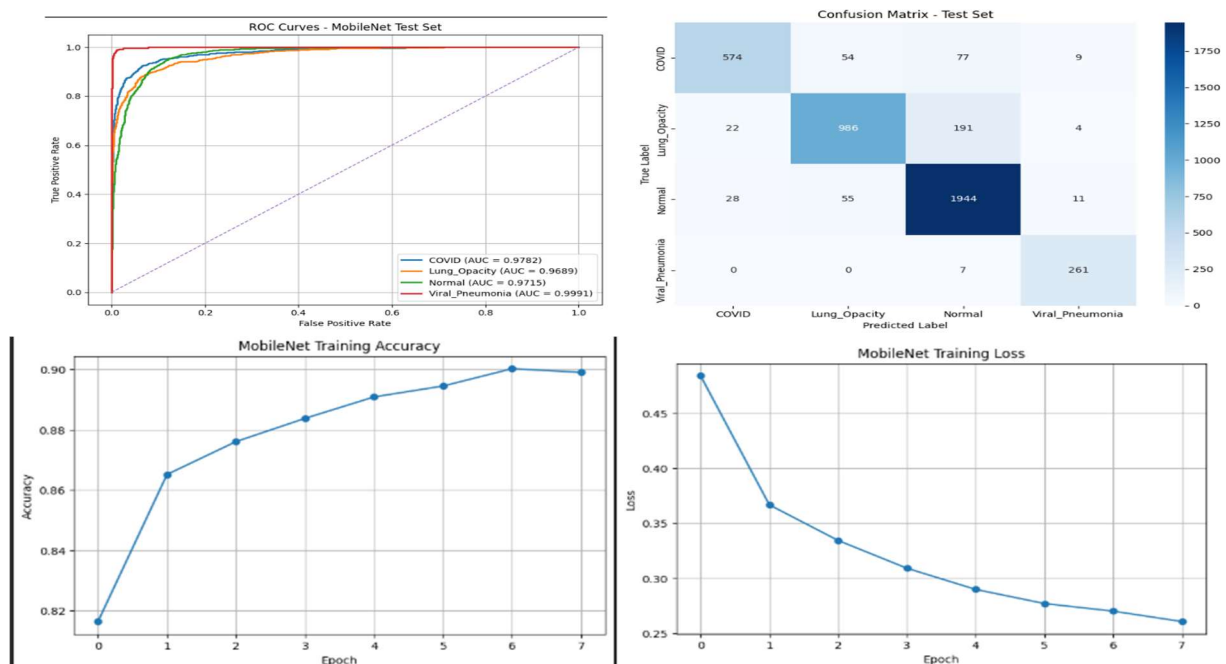
- Models used (e.g., Random Forest, CNN, Logistic Regression)
- Validation techniques (e.g., k-fold, stratified hold-out)
- Evaluation metrics (accuracy, AUC, precision/recall, etc.)
- Regularization, feature selection, hyperparameter tuning

Two deep learning models were implemented and compared. The first model was a custom Convolutional Neural Network (CNN) consisting of convolutional, pooling, and fully connected layers. The second model employed transfer learning using MobileNet. Both models were evaluated using stratified 5-fold cross-validation. Model performance was assessed using accuracy, precision, recall, F1-score, confusion matrix analysis, ROC curves, and Area Under the Curve (AUC).

8. Results

- ROC curve, confusion matrix, performance scores
- Comparison of different models, if applicable

The custom CNN achieved a mean cross-validation accuracy of 67.65%, while MobileNet achieved 87.96%, demonstrating a substantial improvement through transfer learning. The final MobileNet model was retrained on the complete training set and evaluated on an independent test set, achieving a test accuracy of 89.15%. The classification report showed strong performance across all classes, while ROC analysis produced a macro-average AUC of 0.9795, indicating excellent discriminative capability.



9. Discussion

- *Interpretation of results*
- *Limitations*

The experimental results demonstrate that transfer learning significantly outperformed the custom CNN architecture. MobileNet achieved both higher accuracy and more stable cross-validation performance. The confusion matrix revealed that most classification errors occurred between COVID-19, Lung Opacity, and Normal cases, which is expected due to similarities in chest radiographic patterns. Viral Pneumonia achieved the highest classification performance among all categories.

10. Conclusions and Future Work

- *Short conclusions*
- *Ideas for future improvement or next steps*

This project demonstrated the effectiveness of deep learning for automated chest X-ray classification. MobileNet achieved high accuracy and excellent ROC-AUC performance, substantially outperforming the baseline CNN model. Future work may include testing additional transfer learning architectures such as EfficientNet or DenseNet, incorporating data augmentation techniques, and evaluating the model on external clinical datasets to further assess generalization performance.

11. Ethics and Data Privacy

Describe how ethical and privacy considerations were addressed in the project.

- *Confirm that the dataset used is publicly available and permitted for academic or research*
 - *State any licenses or usage restrictions attached to the data or to any third-party code used.*
- If applicable, include a reference to the data source's license or terms of use.*

The project used a publicly available chest X-ray dataset obtained from Kaggle. No personally identifiable patient information was accessed or processed. The study was conducted solely for educational and research purposes. Dataset usage complied with the terms and conditions provided by the dataset source.

12. Code and Reproducibility

Describe how ethical and privacy considerations were addressed in the project.

- *Confirm that the dataset used is publicly available and permitted for academic or research*
 - *State any licenses or usage restrictions attached to the data or to any third-party code used.*
- If applicable, include a reference to the data source's license or terms of use.*

The project was implemented in Python using TensorFlow, Keras, NumPy, Pandas, Scikit-learn, Matplotlib, and Seaborn. Experiments were conducted using Google Colab with GPU acceleration. All preprocessing, training, evaluation, and visualization steps were implemented in reproducible notebook cells.

13. References

- [1] T. Rahman, A. Khandakar, Y. Qiblawey, S. Tahir, S. Kiranyaz, M. Kashem, et al., “Exploring the Effect of Image Enhancement Techniques on COVID-19 Detection Using Chest X-ray Images,” *Computers in Biology and Medicine*, vol. 132, 2021.
- [2] Tawsifur Rahman, “COVID-19 Radiography Database,” Kaggle Dataset. Available: <https://www.kaggle.com/datasets/tawsifurrahman/covid19-radiography-database>
- [3] A. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [4] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” 2015. Available: <https://www.tensorflow.org>
- [5] F. Chollet et al., “Keras,” 2015. Available: <https://keras.io>
- [6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [7] T. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [8] W. McKinney, “Data Structures for Statistical Computing in Python,” *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010.